

5.1

OSI 七层模式

open system Interconnection

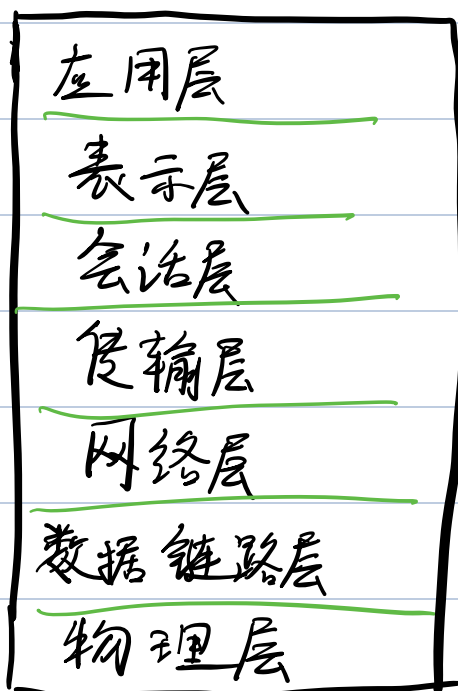
网络通信分层

参考模型

实现

不同厂商设备之间

互联与通信



面向用户/程序 (HTTP)

数据格式转换 (ASCII, UTF-8)

会话管理

端到端通信 (TCP)

实现不同网络间的数据转发 (IP地址)

比特流封装成帧

比特流0/1的传输 (网线, 光纤)

5.2

TCP/IP

四层模型

应用层

传输层

网络层

网络接口层

应用层 + 会话层 + 表示层

传输层

网络层

物理层 + 数据链路层

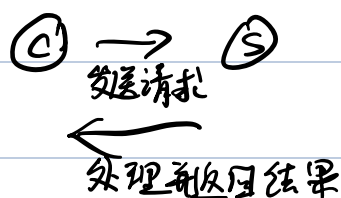
(更接近实际)

5.3

CS 模型

client - server 模型
客户端 服务器

- 集中式网络通信模型



P2P 模型

peer to peer

- 每个节点既是客户端又是服务器
- 节点对等，分布式

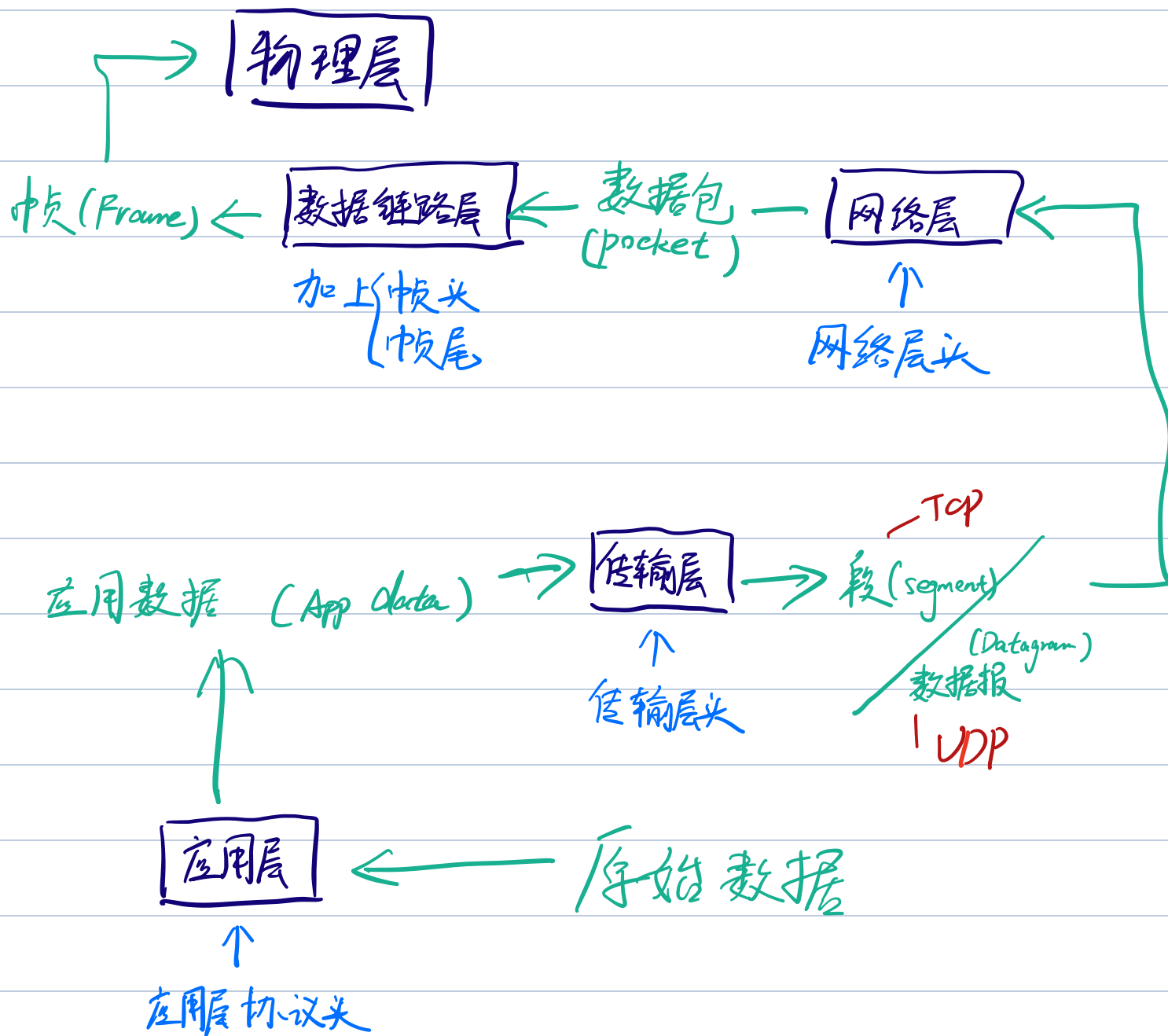
5.4

各层的作用

	作用	协议
应用层	为用户和程序提供网络服务接口	HTTP, MQTT
传输层	端到端通信 <u>源主机</u> → <u>目标主机</u> 数据	TCP UDP
网络层	不同网络间数据转发与寻址	IP

数据封装 流程

5.5 (发送端)

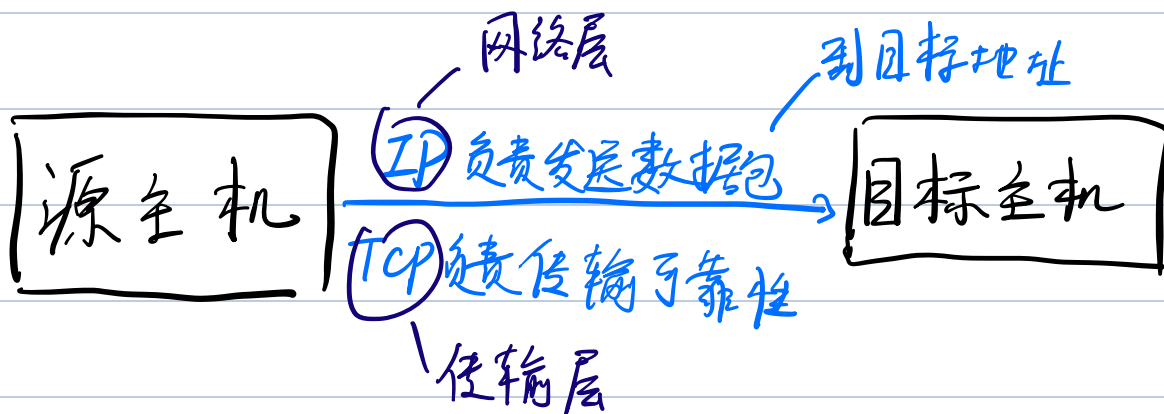
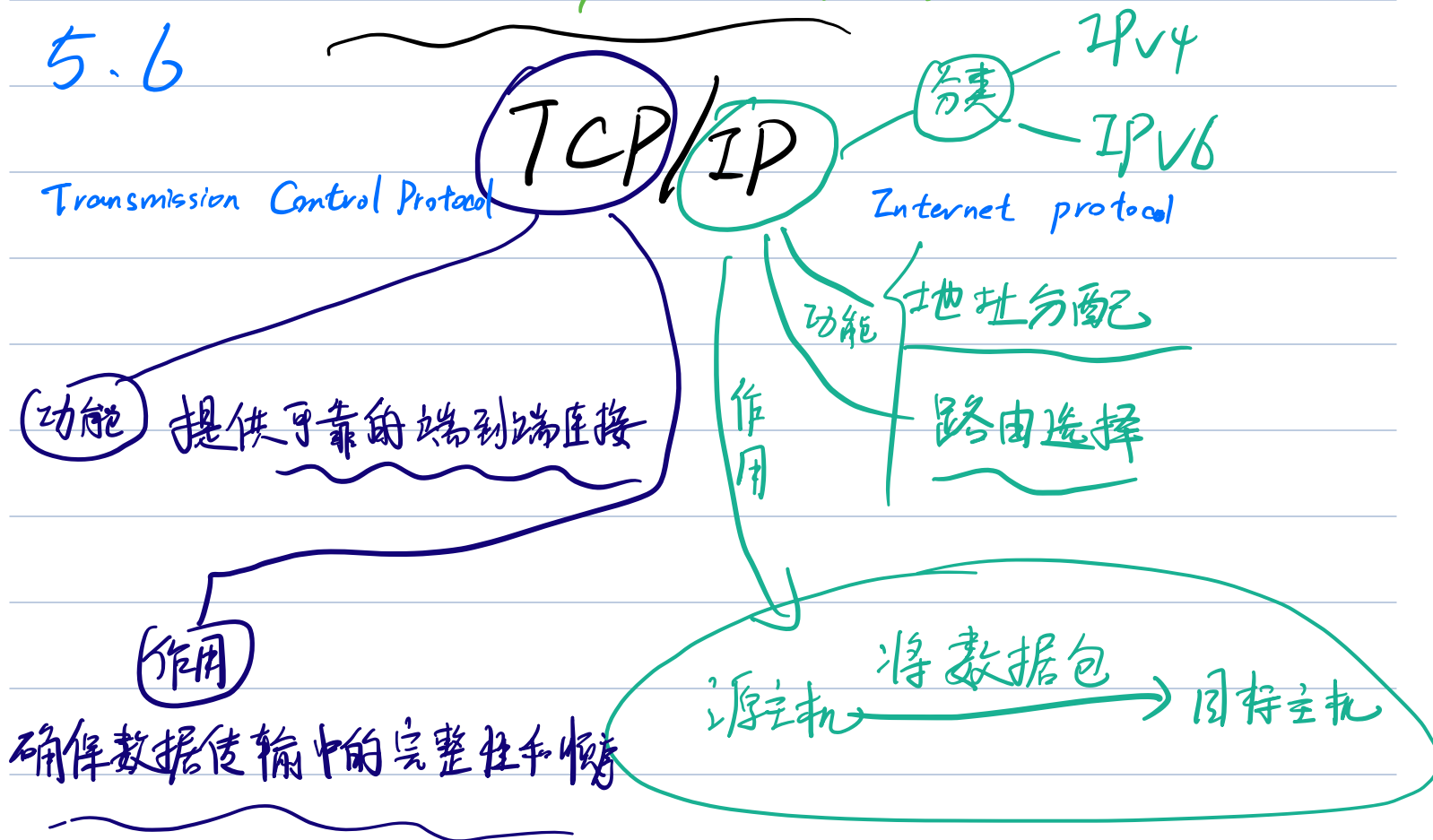


封装: 发送端从应用层到物理层逐层加头(尾), 形成帧

解封装: 接收端从物理层到应用层逐层解头/尾, 恢复原始数据

— 组成全球互联网/局域网通信的基础协议

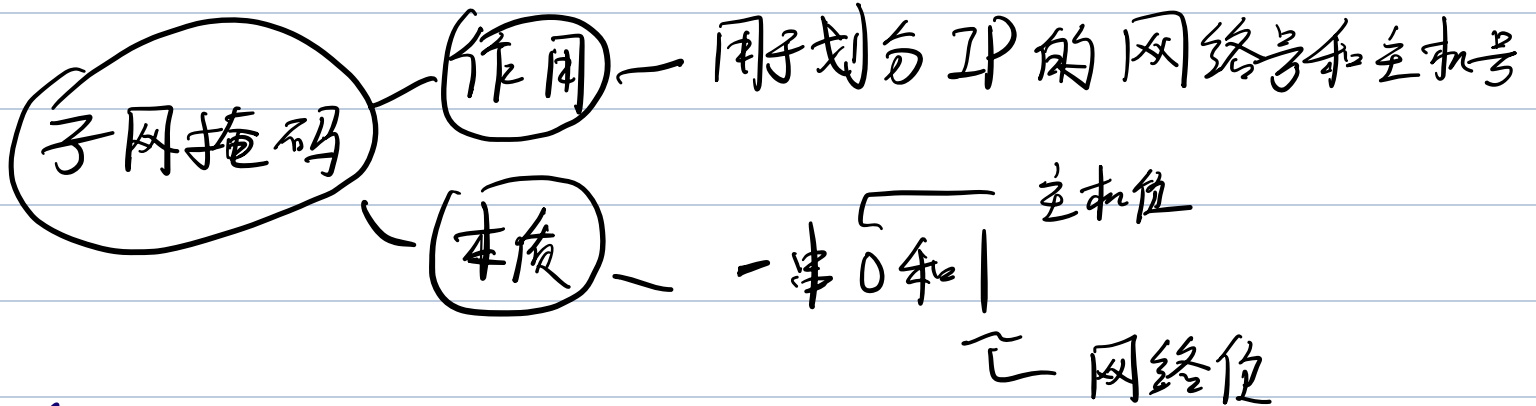
5.6



5.7

$IP = \text{网络号} + \text{主机号}$

└ 相同: 属于同一子网



网络号 = $IP \& \text{子网掩码}$
主机号 = $IP \& (\sim \text{子网掩码})$

5.8

(Media Access Control)
MAC 地址

IP 地址

出厂时固定

本质

网卡的物理地址

网络中的逻辑地址

工作层

数据链路层

网络层

数据帧传输

作用

同一局域网内

唯一标识一台设备

标识设备在网络中的位置, 用于跨

网络通信 数据包传输

长度

48位

IPv4 : 32位

IPv6 : 128位

IP/MAC 通信中的配合:



IP负责跨网“去哪”，MAC负责网内“送给谁”。

跨网络路由

局域网内通信

5、10

TCP建立联接的过程

三次握手

目的

确认双方都能发送/接收数据

同步序号

步骤:

① 发送 SYN(同步)

③ ACK

Client

Server

⇒ 连接成功!

表示要建立连接,携带 ISN(初始序号)

双方传递数据

② 返回 SYN + ACK (确认)

确认①的SYN收到

表示服务器也要建立连接, 也发送ISN

每个 ACK = 确认的 SYN 所带的 ISN + 1

5.11

为什么是三次握手不是两次?

确保双方都能收到

客户端没有确认服务器序列号是否收到

可能把旧 SYN-ACK 报文当作新连接

5.12

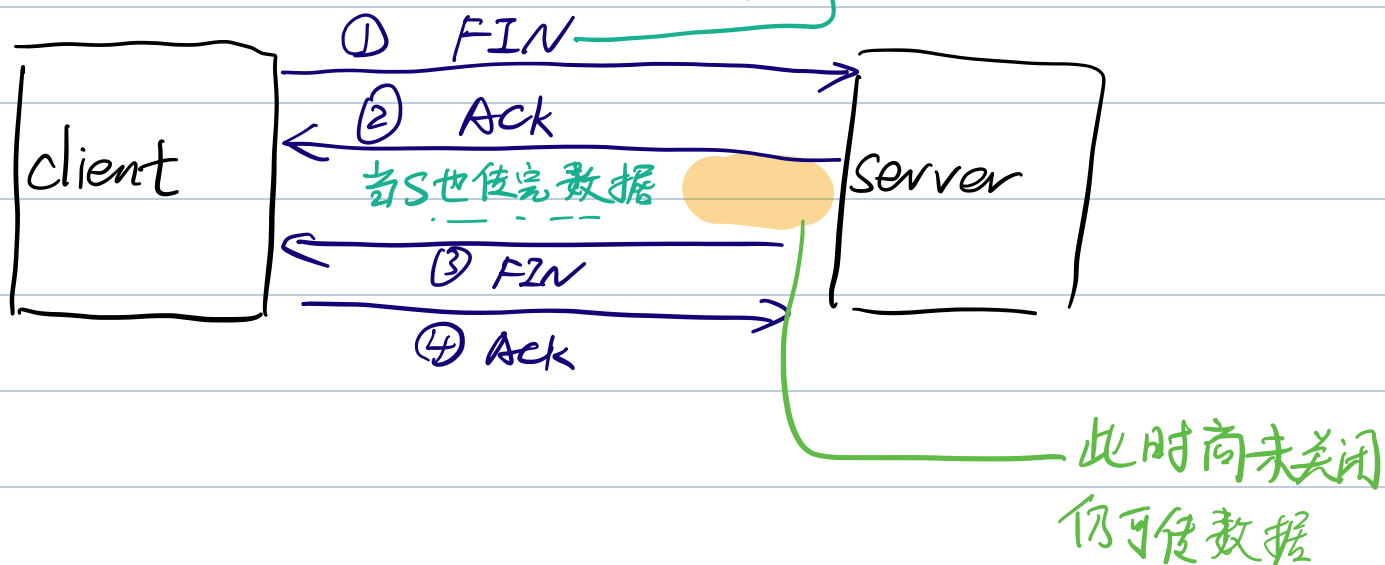
TCP 四次挥手

场景 - TCP 断开连接

目的

保证双方的数据都传输完后再断开

过程：以 client 主动断开为例



5.13

二/三次挥手? X

可能 server 还有未发完的数据
会数据丢失

四次：各自确认已传完 ✓

5.14

TCP 可靠性体现

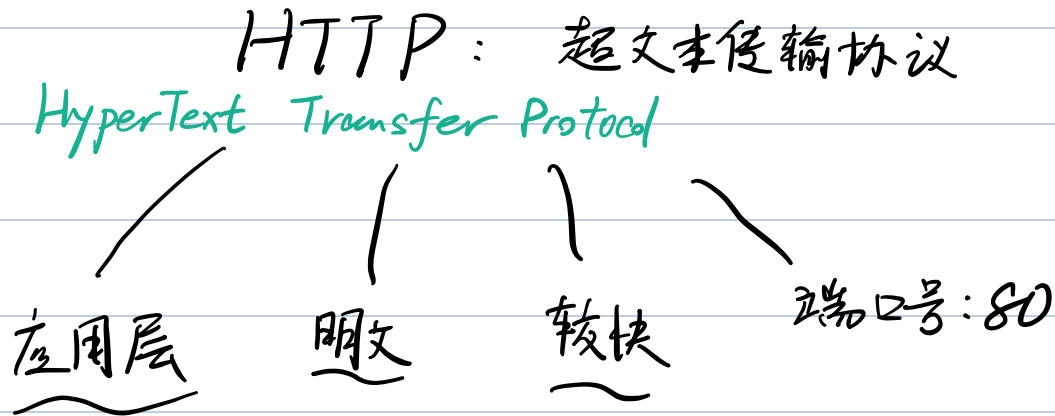
^{确认应答}
序列号、Ack、校验和 → 数据完整性

序列号 → 传输有序性

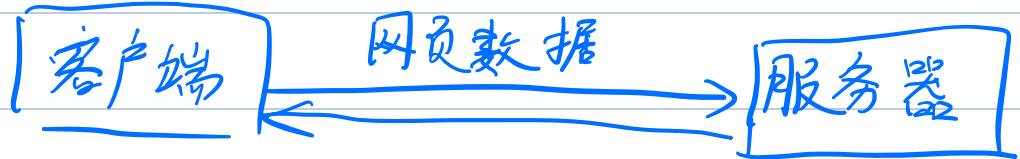
RTO 超时重传 → 可靠交付

流量控制 → 平衡网络负载

5.15

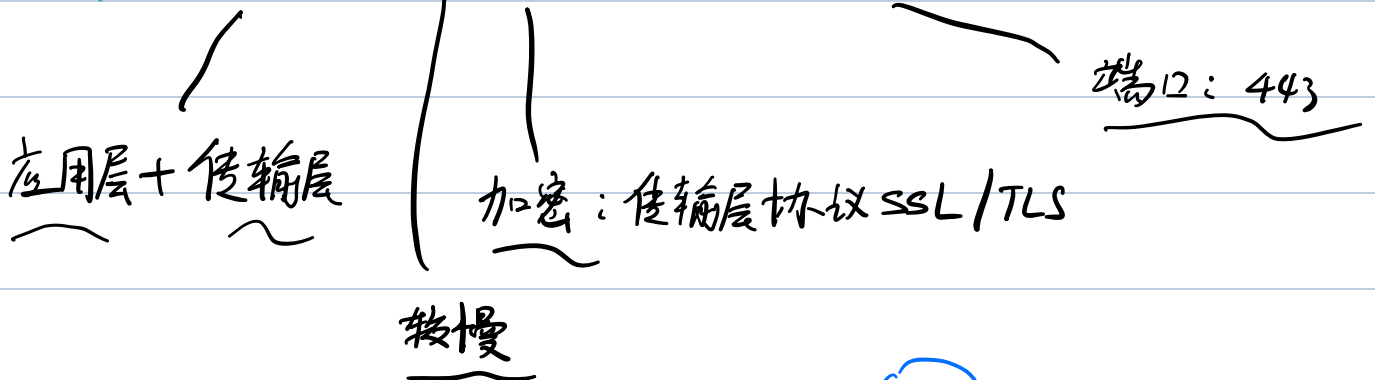


作用:

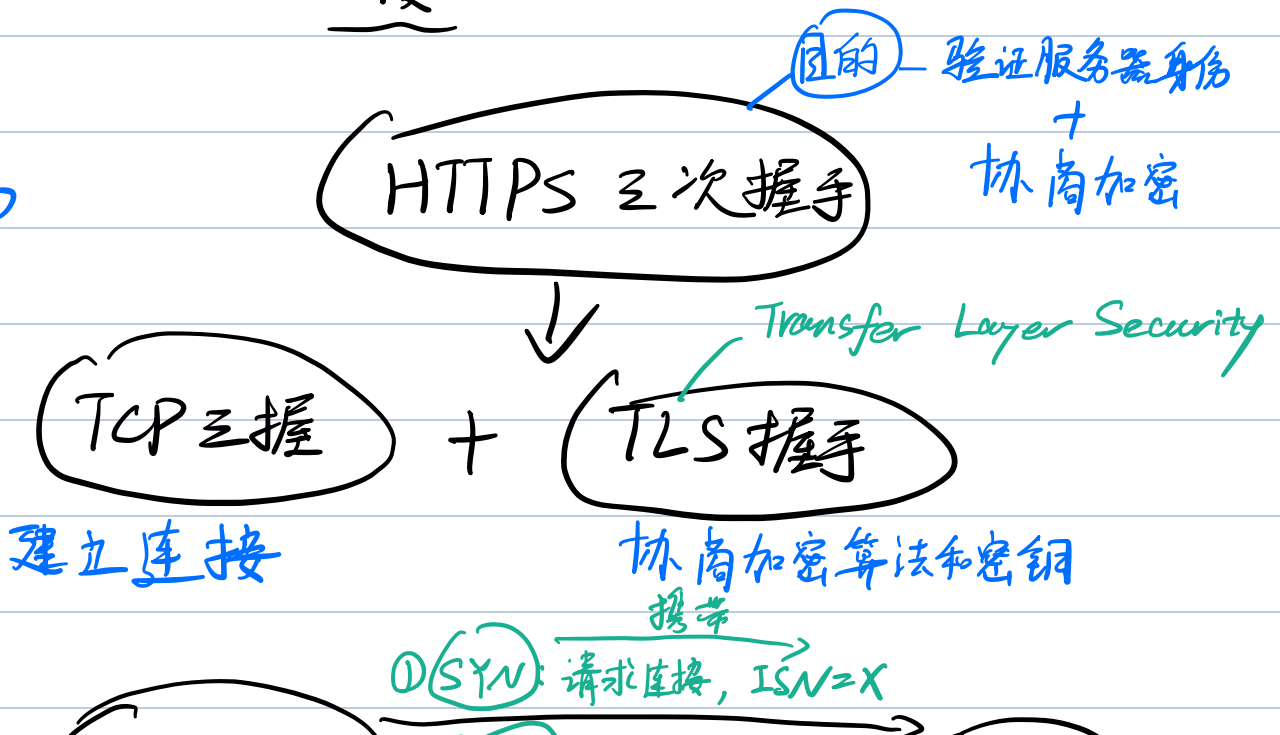


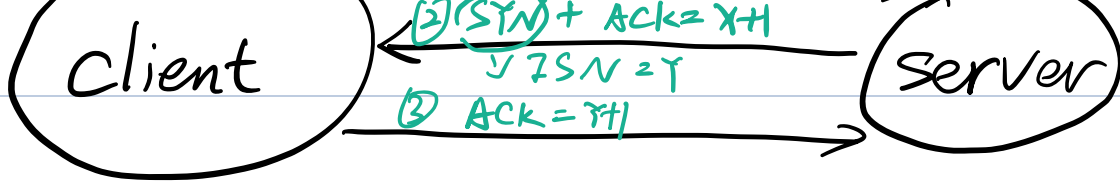
HTTPS: HTTP安全版 = HTTP + TLS/SSL

HyperText Transfer Protocol Secure



5.16

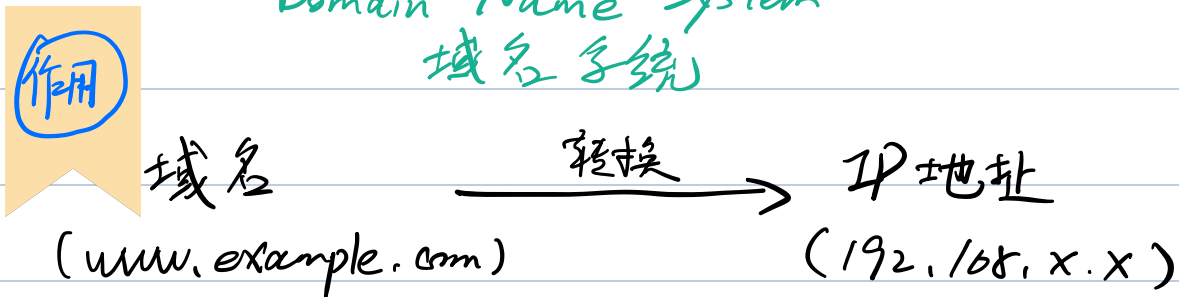




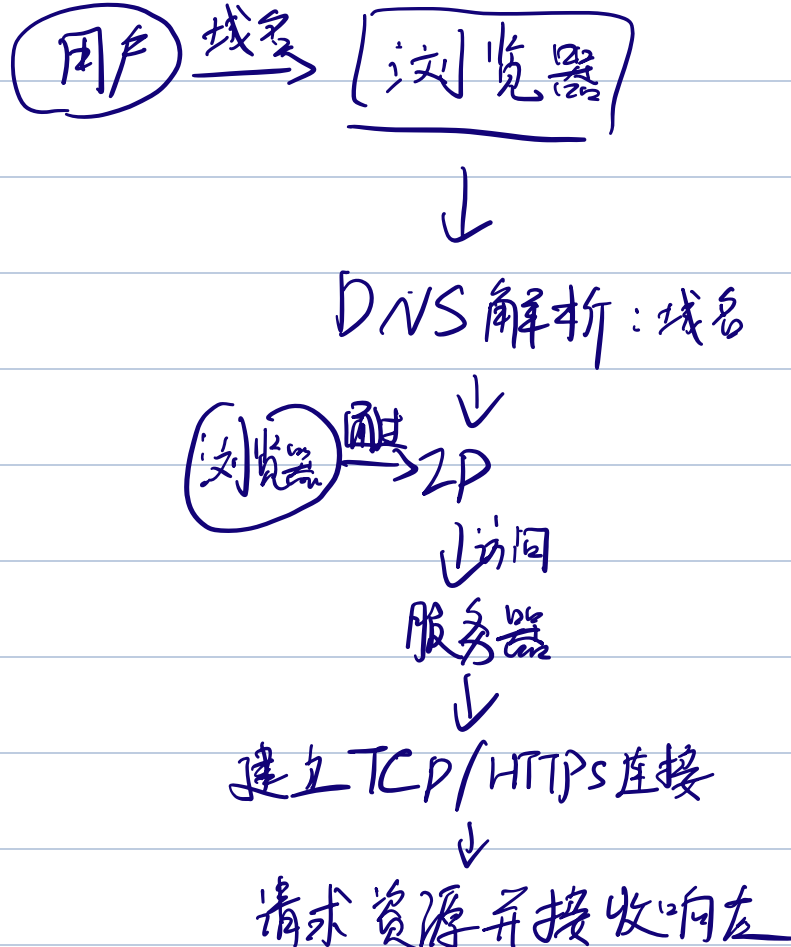
5.17

DNS

Domain Name System
域名系统



互联网访问过程：



5.18

ARP: 地址解析协议

Address Resolution Protocol

功能

IP 地址 $\xrightarrow[\text{ARP}]{\text{转换}}$ MAC 地址

工作层

网络层和数据链路层

作用

局域网内确认目标设备的硬件地址

(局域网内设备间通信)

5.19

短连接 高并发场景

连接快, 断开快

同一时间有大量请求到达

主要问题

⇒ 频繁连接断开 → 系统开销大

⇒ 文件描述符, 端口, 内存等资源易耗尽

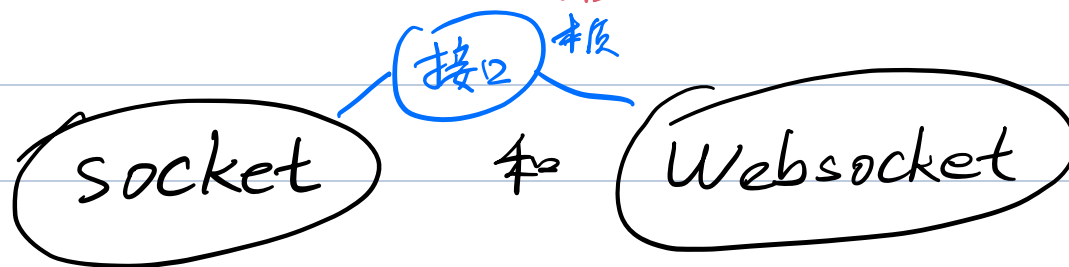
⇒ 进/线程切换成本高

应对: epoll + 线程池 + 限流机制

Linux 中的 I/O 多路复用机制

└ 一个线程同时监听多个 socket

5.20



传输层协议

应用层协议

基于 TCP/UDP

基于 TCP

底层接口,
设备/进程间通信

浏览器 $\rightleftharpoons$ 服务器
间通信

传输比特流

传输帧

无握手

HTTP(S) 握手

任意网络通信
游戏, 文件传输

一次连接后保持实时通信

聊天, 推送...
web 实时通信

5.21

TCP 流量控制

控制谁? 发送端. 的发送速率
为啥? 防止接收端收不过来而丢失~

实现? 窗口机制

接收窗口: TCP 中收端返回的

动态滑动调整 ACK 包中会有一个接收窗口大小

表示剩余缓冲区大小

当窗口太小，发送端会暂停发送

5.23

TCP 拥塞控制



过多的数据导致网络性能下降

动态调整发送速率 → 控制网络负载 → 避免网络拥塞

拥塞控制

① 慢启动

刚连接时，发送方的拥塞窗口为一个MSS(最大报文段)

每经过一个RTT(Round-Trip Time)回路延迟，

窗口大小增加一倍

发一个包并接收ACK的时间

② 拥塞避免

当拥塞窗口到达“慢启动阈值”



进入拥塞避免阶段



每过一个RTT，窗口只加一个MSS

避免快速过载

③ 快速重传

一个包丢失

当发送端三次收到同一个ACK，立即重发丢包

④ 快速恢复

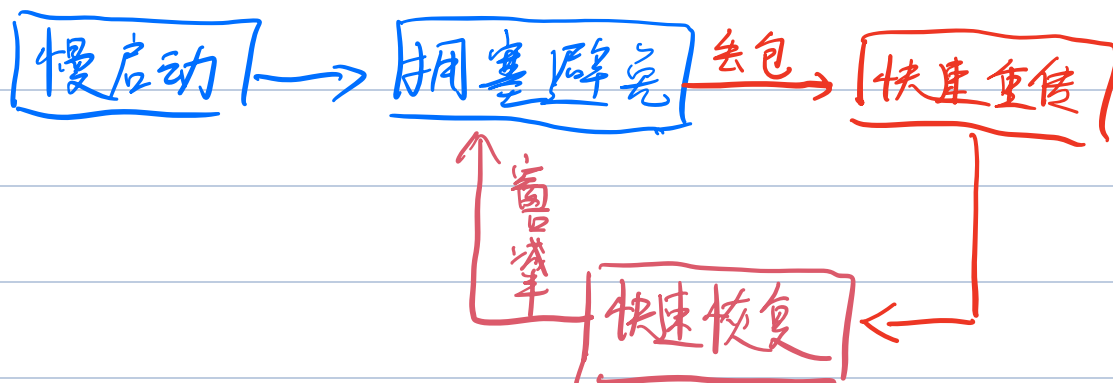
当有丢包触发快速回传时



窗口减半



进入拥塞避免阶段



5.23

TCP 粘包

流式传输：不区分消息边界

发送过多：可能将多个包合并一起发送 (数据合并)

可能导致接收方无法正确区分

TCP 拆包

TCP 按照 MTU (最大传输单元) 将数据分段发送

当包过大, 大于 MTU → 拆包发送

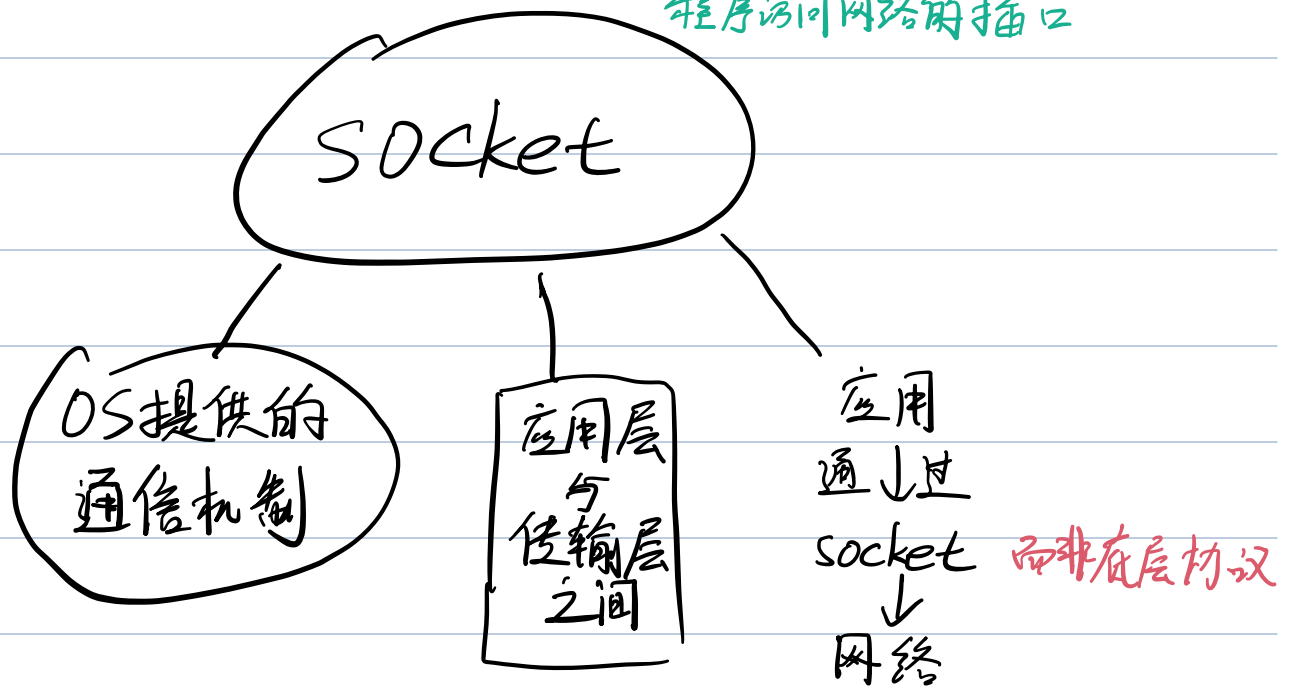
解决：① 固定长度包

② 特殊分隔符

5.24

到底是什么?

程序访问网络的插口



Socket 通信“五元组”

协议 + 源IP + 源端口 + 目的IP + 目的端口

常见类型:

① TCP Socket

- 面向连接: 点对点

- 可靠传输

- 典型应用: HTTP(S)

FTP: 文件传输协议

SSH: 安全外壳协议

- 三次握手

远程登录, 控制服务器

• 传字节流

② UDP socket

• 无连接 : 广播

• 不保证可靠

• 快速, 开销小

典型应用: 视频、语音、实时通信

• 发送数据包

• 可能丢包, 乱序

5.25

Socket 网络编程

{ Socket: 网络通信的端点
 TCP (面向连接)
 UDP (无连接)
 网络编程: 通过 Socket API 在应用程序中网络通信

c/s

(1) 创建 Socket: `socket()`

指定通信协议

{ `SOCK_STREAM`: TCP
 `SOCK_DGRAM`: UDP

T
C
P
为
创

S — (2) 绑定 IP 与端口: `bind()`

S — (3) 监听连接: `listen()`

S — (4) 接收连接, 并返回一个 socket: `accept()`

C — (5) 连接服务器: `connect()`

例

指定 IP 与 端口

s/c

(6) 数据传递: `send()`, `recv()`
`write()`, `read()`

(7) 关闭连接: `close()`

